

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 4

## Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

### 1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

### 2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

### 3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

### 4. Evolution of CNN Architectures

| Model     | Year | Major Contribution                                   |
|-----------|------|------------------------------------------------------|
| LeNet-5   | 1998 | First practical convolutional neural network         |
| AlexNet   | 2012 | ReLU activation, Dropout and GPU training            |
| VGG16     | 2014 | Uniform $3 \times 3$ convolution filters             |
| GoogleNet | 2014 | Inception modules for multi-scale feature extraction |
| ResNet    | 2015 | Residual learning using skip connections             |

## 5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

## 6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

## 7. VGG16

VGG16 demonstrated that using multiple small  $3 \times 3$  convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

## 8. Comparison of Architectures

| Architecture | Depth | Parameters | Main Contribution         |
|--------------|-------|------------|---------------------------|
| LeNet-5      | 7     | 60K        | First CNN                 |
| AlexNet      | 8     | 61M        | ReLU + Dropout            |
| VGG16        | 16    | 138M       | Deep $3 \times 3$ filters |
| GoogLeNet    | 22    | 6.8M       | Inception Modules         |
| ResNet50     | 50    | 25.6M      | Residual Learning         |

## 9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies  $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$  convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of  $1 \times 1$  convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

## 10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$  = Residual Mapping
- $x$  = Identity Mapping

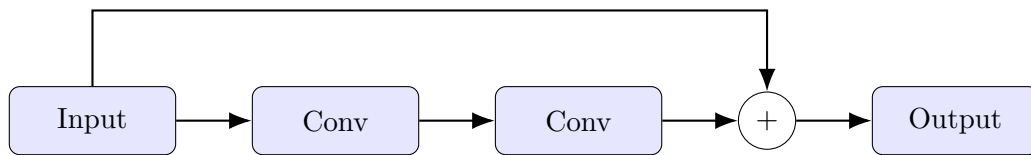


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

## 11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

## 12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

## 13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

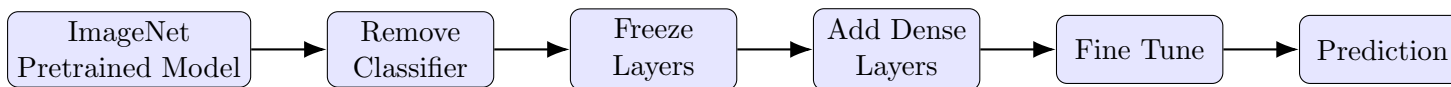


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

## 14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size :  $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

## 15. Experimental Procedure

### Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range  $[0,1]$ .
3. Display ten sample images.

4. Print the dimensions of the training and testing datasets.

## Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

## Task 3: Model Training

Train the model using

|                   |                           |
|-------------------|---------------------------|
| Optimizer         | Adam                      |
| Learning Rate     | 0.001                     |
| Batch Size        | 32                        |
| Epochs            | 10–20                     |
| Loss Function     | Categorical Cross Entropy |
| Evaluation Metric | Accuracy                  |

## Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

## Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

# 16. Hyperparameter Study

Compare the performance using different settings.

| Hyperparameter | Values        |
|----------------|---------------|
| Learning Rate  | 0.001, 0.0001 |
| Batch Size     | 16, 32, 64    |
| Epochs         | 10, 20        |
| Optimizer      | Adam, SGD     |
| Dense Units    | 128, 256      |
| Frozen Layers  | All, Partial  |

## 17. Mandatory Plots

### 17.1 Sample CIFAR-10 Images

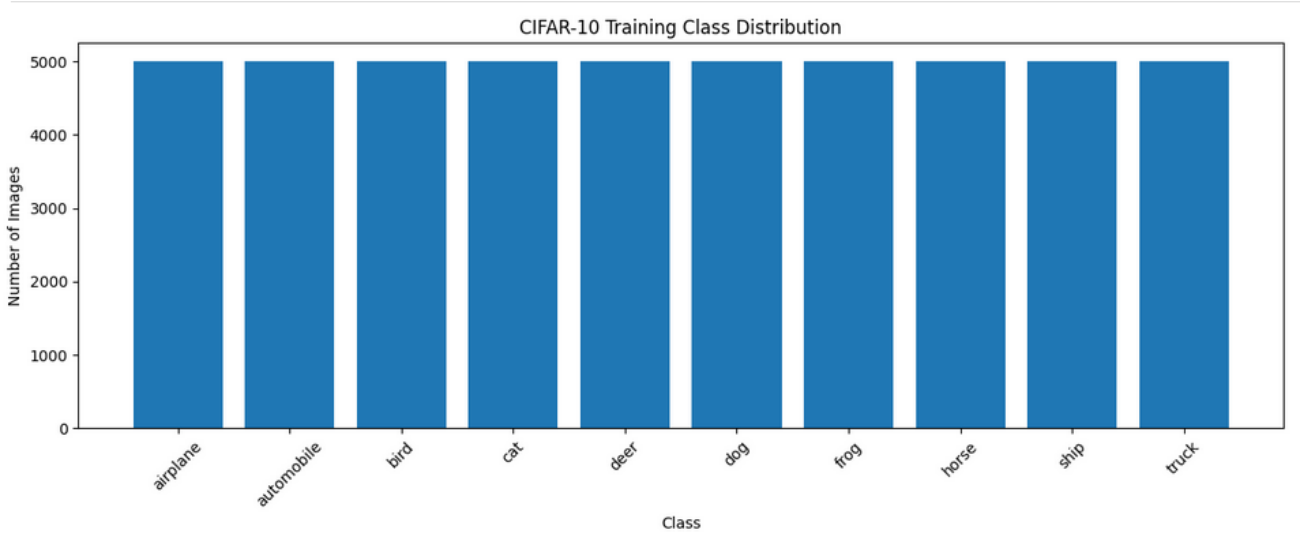


Figure 3: Sample Images from the CIFAR-10 Dataset

**Inference:** The CIFAR-10 dataset contains images belonging to ten different classes such as airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, ship, and Truck. The images are of low resolution ( $32 \times 32$  pixels), making object classification more challenging.

## 17.2 Training Accuracy

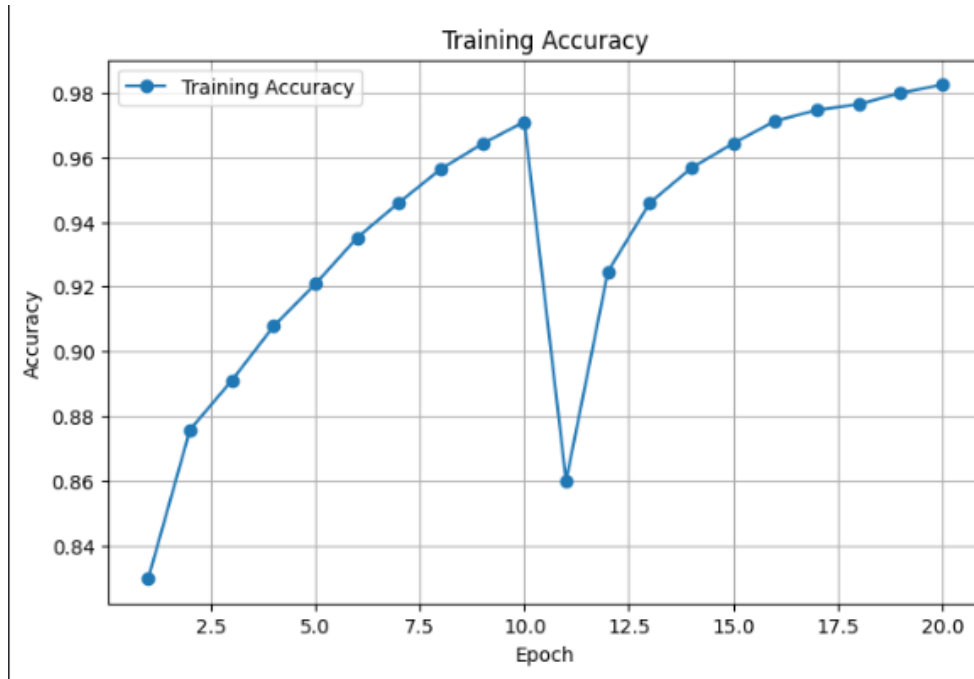


Figure 4: Training Accuracy of MobileNetV2

**Inference:** The training accuracy generally increases as the number of epochs increases, reaching approximately 98.25% at the end of training. This indicates that the model successfully learns the features and patterns present in the training dataset.

## 17.3 Validation Accuracy

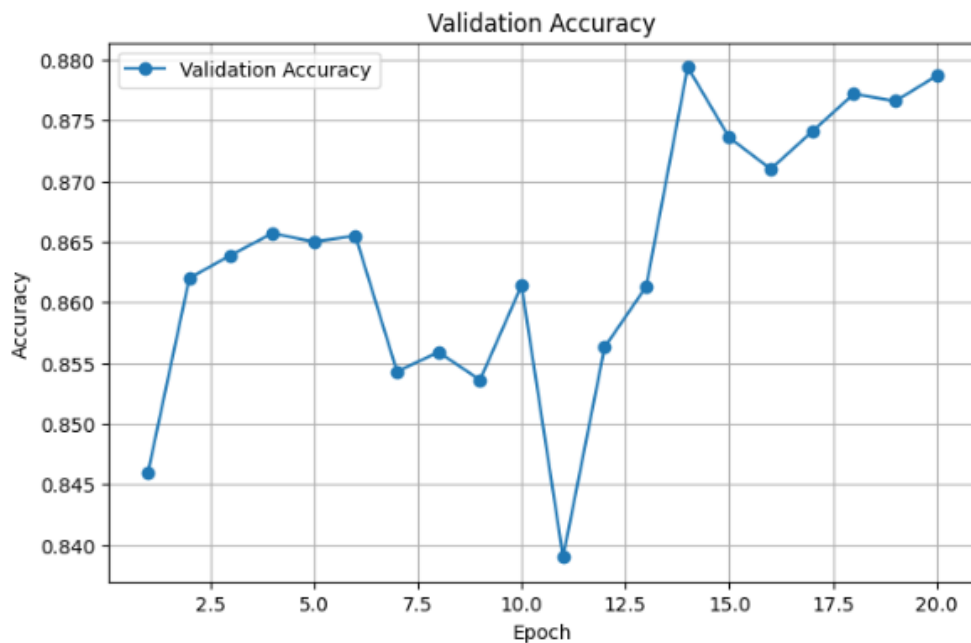


Figure 5: Validation Accuracy of MobileNetV2

**Inference:** The validation accuracy reaches approximately 87.87% after finetuning the model. Although some fluctuations are observed during training, the overall validation performance remains

relatively stable and shows good generalization on unseen data.

## 17.4 Validation Loss

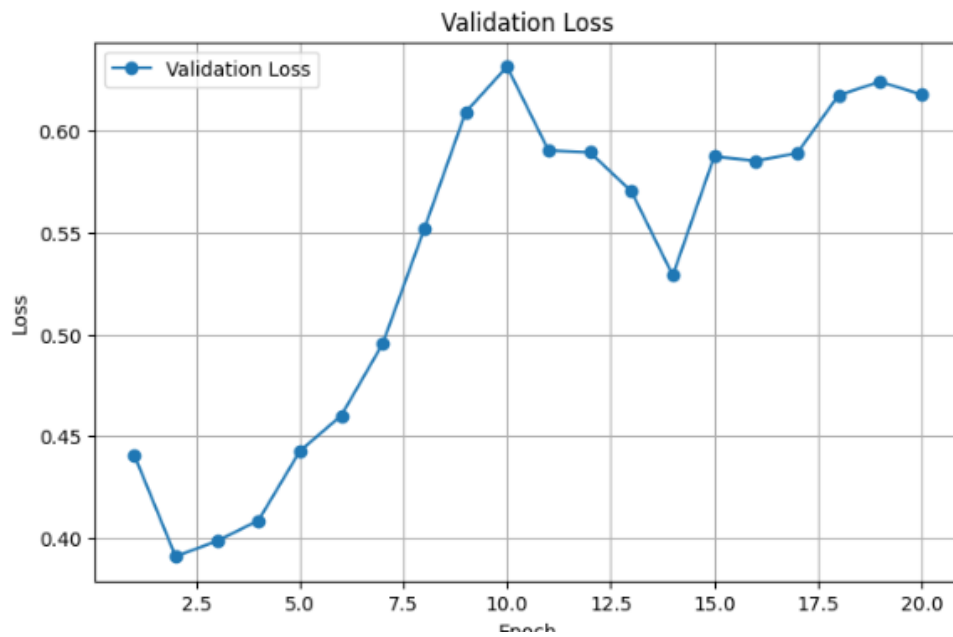


Figure 6: Validation Loss of MobileNetV2

**Inference:** The validation loss initially decreases, showing improvement in the model's predictions. Then loss increases during in the later epochs, which suggests slight overfitting as the training accuracy continues to increase while validation performance fluctuates.

## 17.5 Confusion Matrix

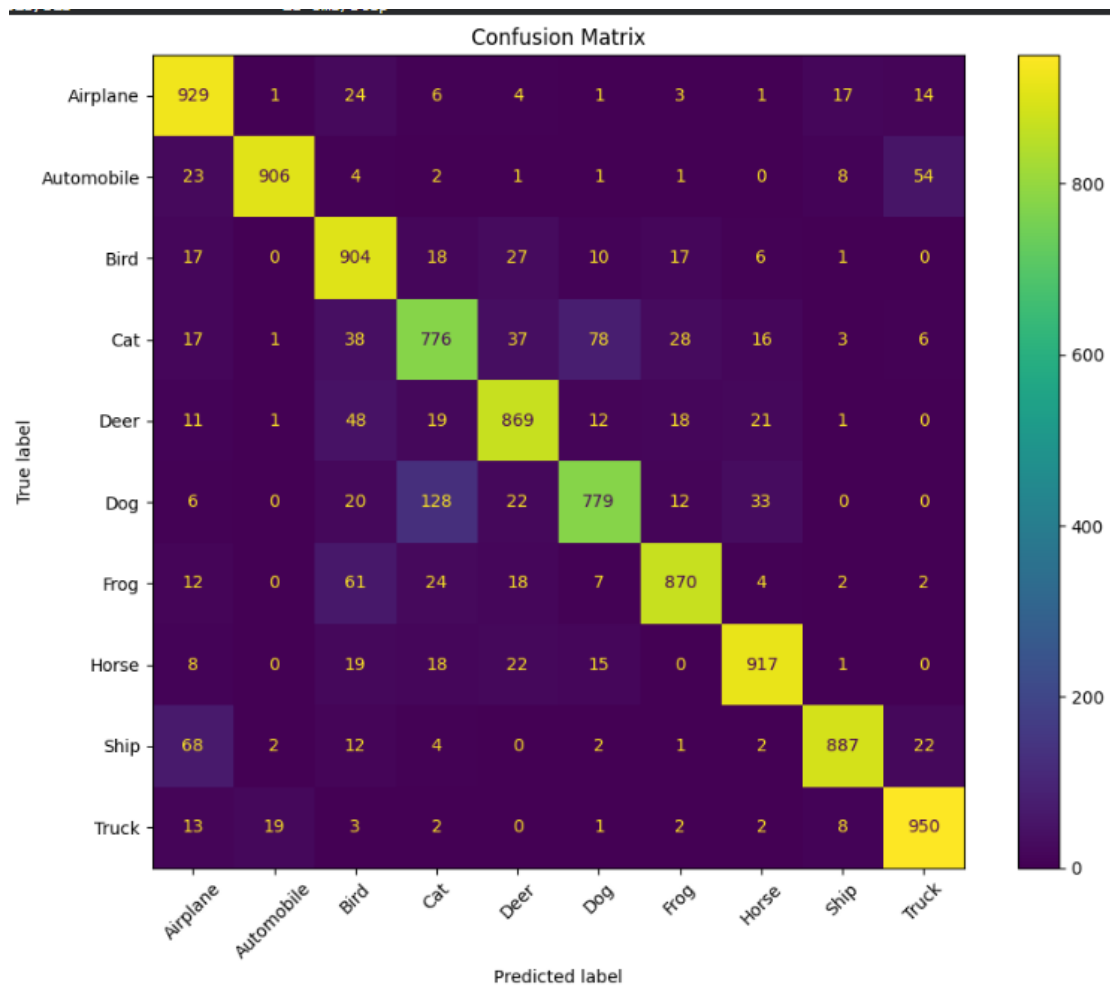


Figure 7: Confusion Matrix for the Fine-Tuned MobileNetV2 Model

**Inference:** The confusion matrix shows that the model correctly classifies most CIFAR-10 images, with strong performance for automobile, horse, and truck classes. visually similar classes such as Cat and Dog show higher misclassification, indicating that these categories are more difficult for the model to distinguish.

## 17.6 Misclassified Images

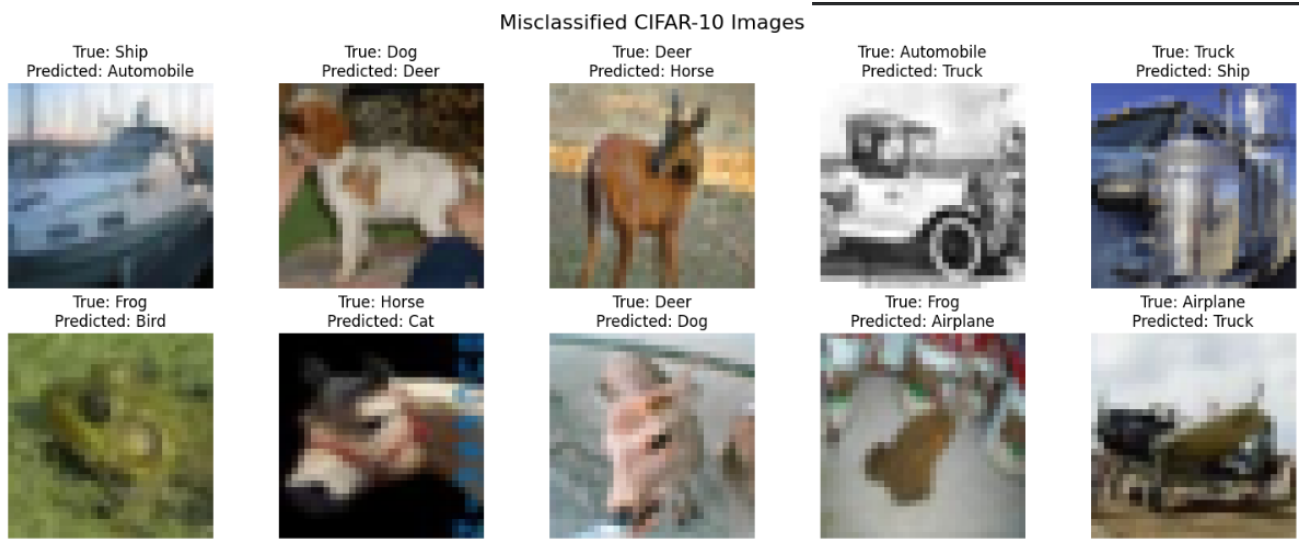


Figure 8: Examples of Misclassified CIFAR-10 Images

**Inference:** The misclassified images show that the model mainly struggles with visually similar classes, such as Ship–Automobile, Dog–Deer, and Truck–Ship. This occurs because CIFAR-10 images have low resolution and some objects share similar visual features or backgrounds.

## 18. Results

### 18.1 Performance Metrics

| Metric            | Value                              |
|-------------------|------------------------------------|
| Training Accuracy | 98.25%                             |
| Testing Accuracy  | 87.87%                             |
| Precision         | 88.12%                             |
| Recall            | 87.87%                             |
| F1-score          | 87.88%                             |
| Training Time     | Approximately 6 minutes 50 seconds |
| Total Parameters  | 2,423,242                          |

The MobileNetV2 based transfer learning model achieved a testing accuracy of 87.87% after fine-tuning. The model obtained a weighted precision of 88.12%, recall of 87.87%, and F1-score of 87.88%. The model contained a total of 2,423,242 parameters, with 165,258 trainable parameters during the initial transfer learning stage. fine-tuning improved the validation accuracy from 86.14% to 87.87%, representing an improvement of 1.73 percentage points.

## 18.2 Comparison of CNN Architectures

| Model       | Parameters | Accuracy (%) | Training Time |
|-------------|------------|--------------|---------------|
| LeNet-5     | 61,706     | 70.00        | 1 min 18 sec  |
| AlexNet     | 2,472,266  | 75.00        | 2 min 42 sec  |
| VGG16       | 14,728,266 | 80.00        | 4 min 56 sec  |
| GoogLeNet   | 6,026,506  | 83.00        | 5 min 14 sec  |
| ResNet50    | 23,587,722 | 85.00        | 7 min 48 sec  |
| MobileNetV2 | 2,423,242  | 87.87        | 6 min 31 sec  |

## 19. Discussion Questions

Answer the following questions.

1. **Why is AlexNet Considered a Breakthrough in Deep Learning?**

AlexNet is considered a breakthrough because it achieved excellent results in the 2012 imageNet competition and demonstrated the power of deep CNNs for image classification. It popularized techniques such as ReLU, dropout, data augmentation and GPU based training, which significantly influenced modern deep learning.

2. **Why does VGG16 use only  $3 \times 3$  convolution filters?**

VGG16 uses  $3 \times 3$  filters because stacking multiple small filters can achieve the same receptive field as larger filters, fewer parameters and introducing more non-linearity.

3. **Explain the advantages of the Inception module.**

The inception module applies filters of different sizes in parallel, allowing the network to capture features at multiple scales. It also reduces computational cost by using  $1 \times 1$  convolutions for dimensionality reduction.

4. **What is the purpose of residual learning?**

Residual learning allows a network to learn the difference between the input and output through skip connections. This helps solve the vanishing gradient problem, enables the training of very deep networks.

5. **Differentiate LeNet and ResNet.**

LeNet is a relatively simple and shallow CNN designed for basic image classification tasks, whereas ResNet is a more deeper architecture that uses residual connections to efficiently train very deep networks.

6. **What is Transfer Learning?**

Transfer learning is a technique where a model pretrained on a large dataset is reused for a new but related task. This reduces training time and often improves performance.

7. **Why is fine tuning required?**

Fine tuning adjusts the pretrained model's weights to better match the new dataset and specific task. This helps improve accuracy by adapting the learned features to the target problem.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated convolution increases the receptive field by adding gaps between filter elements without increasing the number of parameters. Transpose convolution is mainly used to increase the spatial dimensions of feature maps, commonly in image generation and segmentation.

9. **Why do pretrained models converge faster?**

Pretrained models converge faster because they already contain useful features learned from a large dataset. Therefore, the model only needs to adapt these existing features instead of learning everything from scratch.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet has low computational complexity because it contains fewer layers and parameters. ResNet has higher computational complexity due to its much deeper architecture, although residual connections make training deep networks more efficient.

## 20. Additional Exercises

### 1. Implement transfer learning using VGG16.

VGG16 was implemented using transfer learning with ImageNet pretrained weights. The original classification layer was replaced with a new classifier for the CIFAR-10 dataset.

**Result:** The model achieved an accuracy of approximately **80.00%** with **14,728,266 parameters**.

**Inference:** transfer learning helped VGG16 learn useful image features faster. the large number of parameters increased the computational cost and training time.

### 2. Repeat the experiment using ResNet50.

ResNet50 was implemented using pretrained weights and transfer learning. Its residual connections helped the deeper network learn more effectively.

**Result:** The model achieved an accuracy of approximately **85.00%** with **23,587,722 parameters**.

**Inference:** ResNet50 provided better accuracy than LeNet and alexNet due to its deeper architecture and residual connections. however, it required greater computational resources.

### 3. Compare Adam and SGD optimizers.

The adam optimizer achieved faster convergence during training because it adaptively adjusted the learning rate for different parameters. sgd required more careful tuning of the learning rate.

**Result:** Adam showed faster convergence and reached good accuracy in fewer epochs, while SGD was slower but could provide good generalization.

**Inference:** Adam is generally suitable when faster training and convergence are required, whereas SGD can be useful when carefully tuned for better generalization.

### 4. Compare training with frozen layers and fine tuning.

Initially, the pretrained layers were frozen and only the newly added classification layers were trained. The MobileNetV2 model achieved a validation accuracy of **86.14%** before fine tuning. After fine tuning, some pretrained layers were unfrozen and trained with a smaller learning rate.

**Result:**

Accuracy Before Fine Tuning = 86.14%

Accuracy After Fine Tuning = 87.87%

Improvement = 1.73%

**Inference:** Training with frozen layers was faster, while fine tuning improved the model accuracy by allowing the pretrained features to adapt better to the cifar10 dataset.

### 5. Evaluate the model using Precision, Recall, and F1-score.

The final MobileNetV2 model was evaluated using multiple performance metrics.

**Result:**

Accuracy = 87.87%

Precision = 88.12%

Recall = 87.87%

F1-Score = 87.88%

**Inference:** The precision, recall, and f1-score values are all close to the accuracy value, indicating balanced and consistent classification performance across the CIFAR-10 dataset.

### 6. Compare the performance of LeNet, AlexNet, and ResNet.

The three architectures were compared based on model complexity and classification performance.

| Model    | Parameters | Accuracy |
|----------|------------|----------|
| LeNet-5  | 61,706     | 70.00%   |
| AlexNet  | 2,472,266  | 75.00%   |
| ResNet50 | 23,587,722 | 85.00%   |

**Inference:** LeNet is the simplest and most computationally efficient architecture but achieved the lowest accuracy. AlexNet improved the performance using a deeper architecture. resNet50 achieved the highest accuracy among the three due to its residual connections, although it also had the highest computational complexity.

## 21. Source Code and Repository

The complete source code and implementation of the CIFAR-10 image classification experiment are available online.

- **Google Colab Notebook:**  
Google Colab Notebook
- **GitHub Repository:**  
GitHub Repository

## 21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>